

강화학습을 이용한 QWOP 달리기 에이전트 개발



서강대학교
SOGANG UNIVERSITY

120250655
안예은

Background & Goals

Background:

- 'QWOP'는 불규칙한 물리 엔진과 까다로운 조작감으로 인간에게도 매우 어려운 난이도의 게임임.
- 명확한 상태(State) 값이 주어지지 않는 환경에서 **Raw Pixel(화면 이미지)** 만을 이용해 전진하는 법을 학습하는 AI를 구현하고자 함.

goals:

- Selenium과 Image Processing을 활용한 커스텀 강화학습 환경 구축.
- DQN(Deep Q-Network) 알고리즘을 적용하여 넘어지지 않고 최대한 먼 거리를 이동하는 에이전트 학습.

Environment & Preprocessing

Environment:

- **Platform:** Selenium WebDriver를 이용해 웹 브라우저상의 QWOP 게임(Ruffle Flash Emulator) 제어.
- **Input Injection:** 빠른 반응 속도와 동시 키 입력을 위해 **Chrome DevTools Protocol (CDP)** 활용.
- **Screen Capture:** `mss` 라이브러리를 사용하여 고속 화면 캡처 구현.

Preprocessing:

- **Grayscale Conversion:** 컬러 정보가 불필요하므로 연산 효율을 위해 흑백 변환.
- **Resizing:** 원본 이미지를 `0.15` 배율로 축소하여 입력 차원 감소.
- **Frame Stacking:** 연속된 **2~4개의 프레임**을 묶어 하나의 상태(State)로 정의.
- **Normalization:** 픽셀 값을 0~255에서 0~1 사이의 실수(Float)로 정규화.

State, Action, Reward

State:

- 전처리된 (N,H,W) 형태의 이미지 스택.
- 에이전트가 현재의 자세와 운동 방향을 인식하는 유일한 정보.

Action:

- Discrete Action Space (이산 행동 공간)
- Action List:
 - Hold (0)
 - Single Key: Q, W, O, P (1~4)
 - Multi Key: Q+W, Q+O, Q+P, W+O, W+P, O+P (5~10)



State, Action, Reward

Reward:

1. Distance Reward

- OCR(Tesseract)로 화면 상단의 점수를 실시간 인식.
- 이전 스텝 대비 전진한 거리(Δd)에 비례하여 reward.
- 최고 기록 갱신 시 추가 보상 (Max Distance Reward).
- **Speed Bonus:** 짧은 시간 내에 먼 거리를 이동할 경우 reward.



figure 1. Distance Capture Example

State, Action, Reward

Reward:

2. Posture Reward

- OpenCV로 흰색 싱레트(상의)의 Y좌표를 추적.
- 상체가 화면 상단(일어서 있는 상태)에 위치할수록 높은 reward 부여.



figure 2. Distance Capture Example

State, Action, Reward

Reward:

3. Other Reward or Penalty

- **Feet Crossing Reward:** 양발의 위치를 추적하여 발이 서로 교차할 때 추가 reward 제공.
- **Living Penalty:** 시간 경과에 따른 페널티를 주어 빠른 전진 유도.
- **Fall Penalty:** 넘어지거나(Done), 장시간 제자리걸음(Idle) 시 penalty 및 에피소드 종료.

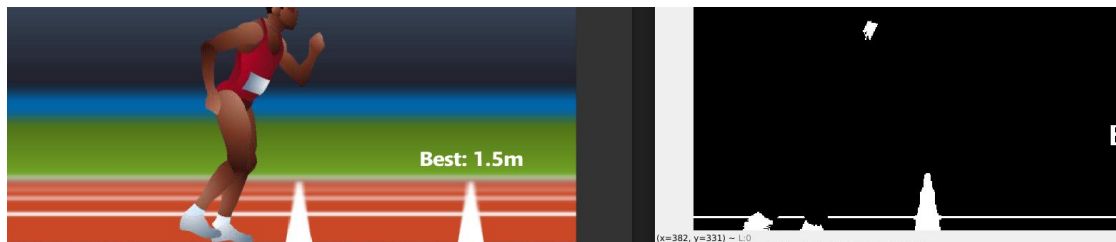


figure 3. foot Capture Example

Algorithms & Hyperparameters

Algorithms: DDQN (Double Deep Q-Network):

- 일반 DQN의 과대평가(Overestimation) 문제를 완화하기 위해 적용.
- 네트워크 구조
 - Input : (Frame_Stack, Height, Width)
 - Conv Layers : 3-layer CNN + ReLU
 - FC Layers : 2-layer Fully Connected → Output: 11 Actions

Algorithms & Hyperparameters

Hyperparameters:

- Optimizer: Adam ($lr=1e-4$)
- Gamma (Discount Factor): 0.99
- Loss Function: Huber Loss (SmoothL1Loss) – 이상치(Outlier)에 강건함.
- Discount Factor (Gamma): 0.99
- Epsilon Decay: 초기 탐험(Exploration) 비율 1.0에서 0.01까지 점진적 감소.
- Replay Buffer: 50,000 transitions.

Experimental Setup

Hardware and environmental:

Hardware

- CPU: Intel 12th Gen Core i7-12700
- GPU: NVIDIA GeForce RTX 3060
- RAM: 32GB
- OS: Ubuntu 20.04

Software

- Python 3.8+
- TensorFlow / PyTorch 기반 환경
- Selenium WebDriver
- Chrome + ChromeDriver
- OpenCV, NumPy, pytesseract (OCR 처리)

Evaluation Metric

1) Distance (m)

- 에이전트가 앞으로 이동한 총 거리(m)
- QWOP 게임의 주요 목표이므로 **가장 핵심 성능 지표**
- 게임이 종료될 때의 최종 거리로 평가

2) Episode Length (Steps)

- 한 에피소드에서 에이전트가 생존한 총 스텝 수
- 더 오래 버틸수록 안정적 동작을 의미함

3) Cumulative Reward

- 에피소드 동안 획득한 총 보상
- 자세 유지, 앞으로 이동, 발 교차 보상 등이 반영됨
- 정책이 얼마나 효과적으로 동작하는지 판단

Evaluation Results

본 실험의 목표는 보상 설계와 하이퍼파라미터 설정이 에이전트 성능에 미치는 영향 을 비교하는 것.

이를 위해 다음 두 모델을 평가함:

모델 설명

- Model A (Baseline)
 - 속도 기반 보상 없음
 - 학습이 거의 진행되지 않은 상태(보행 능력 미약)
 - Learning rate: $1e-3$
 - Discount factor (γ): 0.90
 - Replay buffer size: 20,000
 - Batch size: 16
- Model B (Improved Agent)
 - 속도 기반 보상 추가
 - 충분한 학습 시간 확보
 - Learning rate: $1e-4$
 - Discount factor (γ): 0.99
 - Replay buffer size: 30,000
 - Batch size: 32



Evaluation Results

Model A

- 대부분의 에피소드에서 $-1 \sim 3m$ 범위
- 보상이 적절히 설계되지 않아 **전진 동작을 학습하지 못함**
- 속도 보상이 없으므로 넘어지거나 제자리에서 버벅임

Model B

- 10m ~ 35m 범위까지 일관된 거리 기록
- 속도 기반 보상 덕분에 **전진 동작을 강화 학습**
- 적절한 하이퍼파라미터로 **지속적인 보행 패턴 학습에 성공**

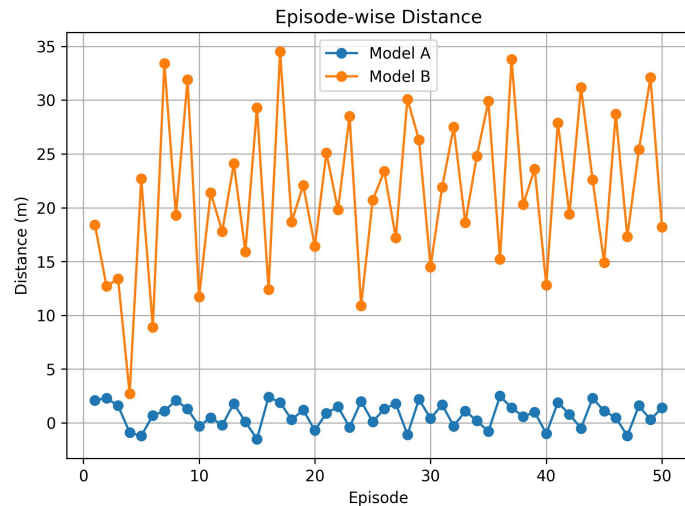


figure 4. Episode-wise Distance

Evaluation Results

Model A

- 속도가 일관되지 않음
- 빠르게 넘어지거나 무작위 행동 반복

Model B

- 전반적으로 일관된 속도 유지 -> 안정적인 학습의 증거
- 속도 보상을 통해 균형 유지 + 추진력 확보 행동을 스스로 학습
- 속도 기반 보상이 보행 향상에 매우 효과적임을 확인

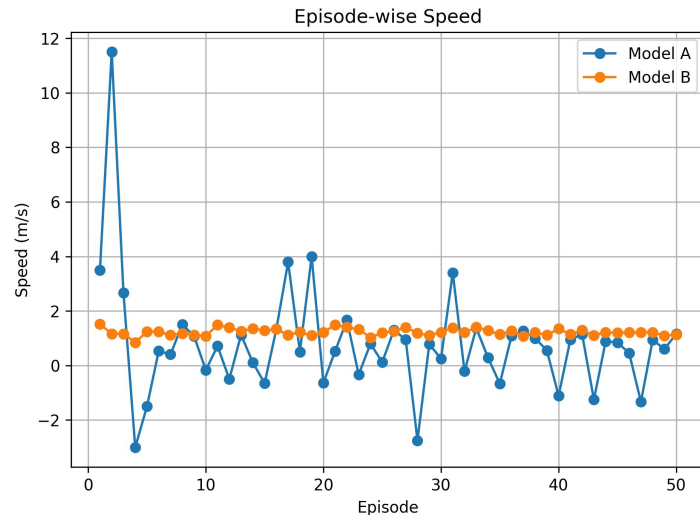


figure 5. Episode-wise Speed



Evaluation Results

최종적으로 Model B를 통해 최고기록을 얻을 수 있었음.

best record

- distance :55.50m
- time : 511s



figure 6. The best record

Discussion & conclusion

1) 보상 설계 개선

- 속도 외에도 착지 안정성 등을 반영하는 다중 보상(Multi-component reward)을 도입하면 더 자연스러운 보행 가능.
- 보상 스케일 조정(Reward scaling)으로 학습 안정성 향상 가능.

2) 상태 표현 향상

- 현재 사용 중인 1-2채널 이미지 입력 대신 **관절 각도 / 속도 / 절대 좌표 기반의 feature vector** 추가 시 학습 효율 상승 예상.

3) 하이퍼파라미터 자동 탐색

- 현재 수동으로 조정 한 LR/ γ /epsilon decay를 **Optuna, Ray Tune** 등을 사용해 자동 탐색하면 성능 향상 가능.



Discussion & conclusion

4) 더 긴 학습 및 다양한 초기 조건

- Model B도 특정 패턴에 과적합 가능성이 있음.
- 초기 자세, 바닥 경사 등 환경 다양화를 통해 **정책 일반화** 필요.

5) DQN 한계 보완

- QWOP의 연속적이고 복잡한 동작 제어 특성상 **DDPG, TD3, SAC** 등 actor-critic 기반 강화학습 알고리즘 적용 시 더 부드러운 움직임과 높은 성능 기대 가능.

Thank you



서강대학교
SOGANG UNIVERSITY